BUỔI 12

BACKEND:

Models/Order.py

backend > models > order.py > OrderItemDB

```python
from sqlalchemy import Column, Integer, String, Float, DateTime, ForeignKey
from sqlalchemy.orm import relationship
from sqlalchemy.sql import func
from database import Base


class OrderDB(Base):
    __tablename__ = "orders"

    id           = Column(Integer, primary_key=True, index=True)
    user_id      = Column(Integer, ForeignKey("users.id"), nullable=False, in
    status       = Column(String(20), nullable=False, default="PLACED")
    total_amount = Column(Float, nullable=False)
    created_at   = Column(DateTime(timezone=True), server_default=func.now())

    items = relationship("OrderItemDB", back_populates="order", cascade="all, del


class OrderItemDB(Base):
    __tablename__ = "order_items"

    id           = Column(Integer, primary_key=True, index=True)
    order_id     = Column(Integer, ForeignKey("orders.id"), nullable=False, inde
    product_id   = Column(Integer, ForeignKey("products.id"), nullable=False)
    product_name  = Column(String(100), nullable=False)
    product_price = Column(Float, nullable=False)
    quantity     = Column(Integer, nullable=False)
    line_total   = Column(Float, nullable=False)
```

Schemas/order.py

backend > schemas > order.py > OrderItemQuantityUpdate

```python
from pydantic import BaseModel, Field, field_validator
from typing import List, Optional

ALLOWED_STATUSES = ["PLACED", "PROCESSING", "SHIPPED", "COMPLETED", "CANCELED"]


class OrderItemCreate(BaseModel):
    product_id: int
    name:       str
    price:      float
    quantity:   int = Field(..., gt=0)


class CheckoutRequest(BaseModel):
    items: List[OrderItemCreate]


class OrderItemRead(BaseModel):
    id:            int
    product_id:    int
    product_name:  str
    product_price: float
    quantity:      int
    line_total:    float

    class Config:
        from_attributes = True
```

```python
class OrderStatusUpdate(BaseModel):
    status: str

    @field_validator("status")
    @classmethod
    def validate_status(cls, v: str) -> str:
        if v not in ALLOWED_STATUSES:
            raise ValueError(f"Invalid status: {v}")
        return v


class OrderItemQuantityUpdate(BaseModel):
    item_id:  int
    quantity: int = Field(..., gt=0)
```

routers/orders.py

```python
from fastapi import APIRouter, Depends, HTTPException, status
from sqlalchemy.orm import Session
from sqlalchemy import desc
from typing import List

from database import get_db
from models.order import OrderDB, OrderItemDB
from schemas.order import (
    CheckoutRequest, OrderRead, OrderItemRead,
    OrderSummary, OrderStatusUpdate, OrderItemQuantityUpdate,
)
from auth.deps import get_current_user, require_admin

router = APIRouter(prefix="/orders", tags=["orders"])

```

FRONTEND:

OrderHistoryPage.jsx

OrderDetailPage.jsx

frontend > src > pages > OrderDetailPage.jsx > default

```jsx
import { useEffect, useState } from 'react';
import { useParams, useNavigate } from 'react-router-dom';
import {
  Container, Typography, Box, Paper, Table, TableBody,
  TableCell, TableContainer, TableHead, TableRow, Chip,
  Button, CircularProgress, Alert, Select, MenuItem,
  FormControl, InputLabel, IconButton, Divider,
} from '@mui/material';
import {
  ArrowBack as BackIcon,
  Add as PlusIcon,
  Remove as MinusIcon,
} from '@mui/icons-material';
import { ordersApi } from '../api/ordersApi';
import { useAuth } from '../auth/useAuth';
```

AdminOrdersPage.jsx

frontend > src > pages > admin > AdminOrdersPage.jsx > default

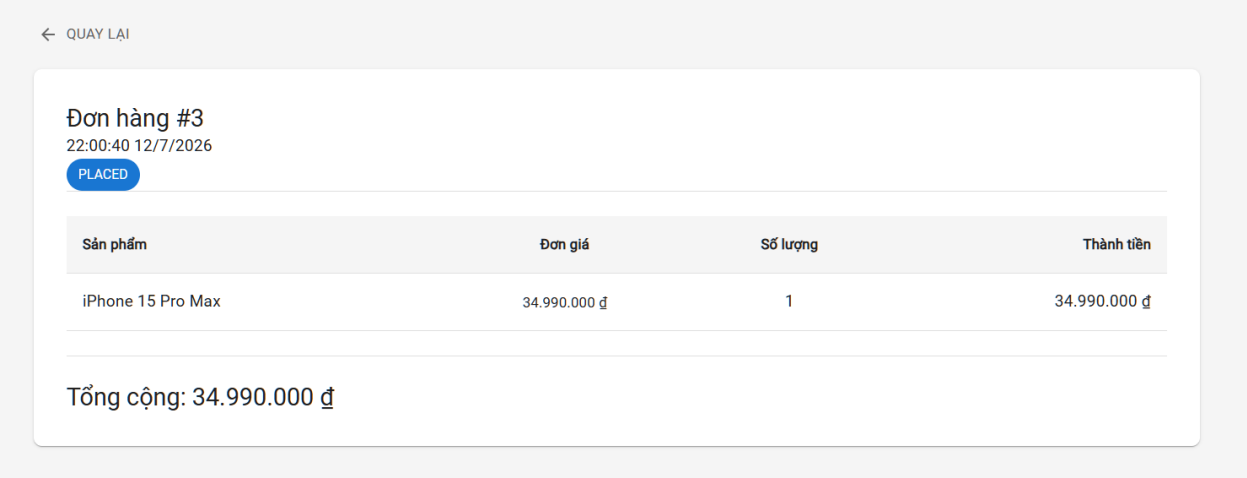
```jsx
import { useEffect, useState } from 'react';
import { useNavigate } from 'react-router-dom';
import {
  Box, Typography, Paper, Table, TableBody, TableCell,
  TableContainer, TableHead, TableRow, Chip, Button,
  CircularProgress, Alert,
} from '@mui/material';
import { Visibility as ViewIcon } from '@mui/icons-material';
import { ordersApi } from '../../api/ordersApi';

const statusColors = {
  PLACED: 'primary', PROCESSING: 'warning',
  SHIPPED: 'info', COMPLETED: 'success', CANCELED: 'error',
};

const AdminOrdersPage = () => {
  const navigate = useNavigate();
  const [orders, setOrders]   = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError]     = useState('');

  useEffect(() => {
    const fetchOrders = async () => {
      try {
        const data = await ordersApi.getAllForAdmin();
        setOrders(data);
      } catch {
        setError('Không thể tải danh sách đơn hàng');
```

routes/PrivateRoute.jsx

frontend > src > routes > PrivateRoute.jsx > default

```jsx
import { Navigate, Outlet, useLocation } from 'react-router-dom';
import { useAuth } from '../auth/useAuth';

const PrivateRoute = () => {
  const { isAuthenticated } = useAuth();
  const location = useLocation();

  if (!isAuthenticated) {
    return <Navigate to="/login" replace state={{ from: location }} />;
  }

  return <Outlet />;
};

export default PrivateRoute;
```

Đơn hàng bên khách hàng:



Quản lý đơn hàng: